Python — ALL Methods, Data-Type Wise (Complete List)

STRING METHODS (28 covered in your notes + standard ones)

#	Method	What it does
1	`upper()`	converts to uppercase
2	`lower()`	converts to lowercase
3	`swapcase()`	swaps upper↔lower for every character
4	`capitalize()`	capitalizes only the first letter of the string
5	`title()`	capitalizes the first letter of **every word**
6	`count(sub, start, end)`	counts occurrences of a substring
7	`replace(old, new, count)`	replaces occurrences of a substring
8	`isalnum()`	True if all characters are letters/digits (no spaces/symbols)
9	`isalpha()`	True if all characters are alphabetic
10	`isdigit()`	True if all characters are digits
11	`isupper()`	True if all letters are uppercase
12	`islower()`	True if all letters are lowercase
13	`isspace()`	True if string is all whitespace
14	`istitle()`	True if string follows title case
15	`index(sub, start, end)`	position of substring; **ValueError** if not found
16	`rindex(sub, start, end)`	position from the **right**; ValueError if not found
17	`find(sub, start, end)`	position of substring; **-1** if not found
18	`rfind(sub, start, end)`	position from the right; -1 if not found
19	`removeprefix(prefix)`	removes given prefix if present
20	`removesuffix(suffix)`	removes given suffix if present
21	`startswith(sub)`	True/False check
22	`endswith(sub)`	True/False check
23	`join(iterable)`	joins elements of iterable using the string as separator
24	`strip(chars)`	removes leading & trailing whitespace/characters

#	Method	What it does
25	`rstrip(chars)`	removes trailing whitespace/characters
26	`lstrip(chars)`	removes leading whitespace/characters
27	`split(sep, maxsplit)`	splits string into a list
28	`rsplit(sep, maxsplit)`	splits from the right

Extra standard string methods (good to know, not always in course notes): `center()`, `ljust()`, `rjust()`, `zfill()`, `format()`, `format_map()`, `encode()`, `expandtabs()`, `partition()`, `rpartition()`, `casefold()`, `isnumeric()`, `isdecimal()`, `isidentifier()`, `isprintable()`, `maketrans()`, `translate()`

LIST METHODS (11 total)

#	Method	What it does
1	`append(x)`	adds one item at the end (as a single element)
2	`extend(iterable)`	adds each item of an iterable individually
3	`insert(i, x)`	inserts item `x` at index `i`
4	`pop(i)`	removes & returns item at index `i` (default: last item)
5	`remove(x)`	removes first matching value `x`
6	`clear()`	empties the entire list
7	`count(x)`	counts occurrences of `x`
8	`index(x, start, end)`	returns position of first occurrence of `x`
9	`sort(key=None, reverse=False)`	sorts the list in place
10	`copy()`	returns a shallow copy
11	`reverse()`	reverses the list in place

TUPLE METHODS (only 2 — because tuples are immutable)

#	Method	What it does
1	`count(x)`	counts occurrences of `x`
2	`index(x, start, end)`	returns position of first occurrence; ValueError if absent

(No append/remove/pop/sort/insert/clear/reverse — immutability blocks all of these.)

DICTIONARY METHODS (11 total)

#	Method	What it does
1	`get(key, default)`	safe access — returns `default`/`None` if key missing
2	`keys()`	returns a view of all keys
3	`values()`	returns a view of all values
4	`items()`	returns a view of all (key, value) pairs
5	`update(other)`	merges another dict / iterable of pairs into this one
6	`pop(key, default)`	removes & returns value for `key`; KeyError if missing (unless default given)
7	`popitem()`	removes & returns the **last inserted** (key, value) pair; KeyError if empty
8	`setdefault(key, default)`	returns value if key exists; else inserts key with default and returns it
9	`fromkeys(iterable, value)`	creates a new dict from an iterable of keys, all mapped to `value`
10	`clear()`	empties the dictionary
11	`copy()`	returns a shallow copy

Also: `dict[key] = value` (add/update entry), `del dict[key]` (delete entry), `{**d1, **d2}` or `d1 | d2` (merge, 3.9+)

SET METHODS (17 total)

Add/Remove:

#	Method	What it does
1	`add(x)`	adds a single element
2	`update(iterable)`	adds multiple elements from an iterable
3	`remove(x)`	removes `x`; **KeyError** if not found
4	`discard(x)`	removes `x`; **no error** if not found
5	`pop()`	removes & returns a **random** element; KeyError if empty
6	`clear()`	empties the set
7	`copy()`	returns a shallow copy

Set algebra (Venn-diagram operations):

#	Method	What it does
8	`union(other)`	all elements from both sets (new set, originals unchanged)
9	`intersection(other)`	common elements only
10	`difference(other)`	elements in this set but not in `other`
11	`symmetric_difference(other)`	elements in either set, but not both
12	`intersection_update(other)`	keeps only common elements (modifies original)
13	`difference_update(other)`	removes elements found in `other` (modifies original)
14	`symmetric_difference_update(other)`	keeps only non-common elements (modifies original)

Relationship checks:

#	Method	What it does
15	`issubset(other)`	True if this set ⊆ other
16	`issuperset(other)`	True if this set ⊇ other
17	`isdisjoint(other)`	True if the two sets share **no** elements

Recall tip: "Every set method that ends in `_update` **modifies the original set**; the plain version (`union`, `intersection`, `difference`, `symmetric_difference`) **returns a new set** and leaves the original untouched."

Quick Method-Count Cheat Card

Type	Total Methods	Mnemonic
String	28 (course) / 40+ (full)	biggest method list — text has the most operations
List	11	add/remove/organize
Tuple	2	only read-only ops: `count`, `index`
Dict	11	access + key-based ops
Set	17	7 basic + 7 algebra + 3 relationship checks

Master recall line: *"Tuple has almost nothing (2) because it's frozen. Set has the most variety (17) because of set algebra. String has the most methods overall because text needs the most processing."*